Lab 12 — Antwoorden met toelichting

Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.

Vraag 1 — Drie ReplicaSets, drie Pods

Antwoord. Drie ReplicaSets (één per versie van het template) en **drie** Pods, allemaal in de meest recente ReplicaSet. `kubectl get rs` toont de laatste op `3 3 3` en de twee andere op `0 0 0`. De oude blijven bewaard omdat ze de geschiedenis **zijn**: `rollout undo` zet gewoon de replica's van een oude ReplicaSet terug op 3 en die van de huidige op 0. `revisionHistoryLimit` (standaard 10) in de spec van het Deployment begrenst hun aantal.

Waarom. Een ReplicaSet op nul kost maar één object in etcd; hem bewaren maakt de terugkeer onmiddellijk, zonder een template te herbouwen.

Nuance. Twee updates die op **hetzelfde** template uitkomen (1.0 → 1.1 → 1.0) hergebruiken de bestaande ReplicaSet: de hash van het template is identiek.

Voorbeeld.

```
kubectl get rs -l app=api   # api-6c7bb54fc5 0 0 0 / api-5c8d86977d 0 0 0 / api-… 3 3 3
```

Vraag 2 — Selector en labels stemmen niet overeen

Antwoord. `The Deployment "api" is invalid: spec.template.metadata.labels: Invalid value: … selector does not match template labels`. De regel: de `selector` van het Deployment **moet** de Pods selecteren die zijn template voortbrengt, anders zou het Pods aanmaken die het niet als de zijne herkent. Aanvaardde het cluster het toch, dan zou de ReplicaSet altijd `0` Pods `app: api` zien, eindeloos Pods `app: backend` aanmaken die hij niet meetelt — een lek van Pods.

Waarom. De relatie Deployment → Pods is geen bezit maar een **selectie**; ze moet van bij de aanmaak kloppen.

Nuance. Omgekeerd zou een met de hand aangemaakte Pod met `app: api` door de ReplicaSet **geadopteerd** worden (en een Pod te veel zou verwijderd worden): labels zijn een contract, geen versiering.

Voorbeeld.

```
kubectl apply -f deployment-fout.yaml 2>&1 | grep -o 'selector does not match template labels'
```

Vraag 3 — De reeks van een rolling update

Antwoord. Met `maxSurge: 25 %` en `maxUnavailable: 25 %` op drie replica's (afgerond: 1 extra, 0 minder) zijn de Events: nieuwe RS `0 → 1`; zodra die Pod klaar is, nieuwe `1 → 2` en oude `3 → 2`; dan oude `2 → 1`, nieuwe `2 → 3`; ten slotte oude `1 → 0`. Op geen enkel moment minder dan drie klare Pods. Drie nieuwe Pods in één keer aanmaken zou zes Pods vragen (overschrijdt `maxSurge`) of drie oude verwijderen voor de nieuwe klaar zijn (overschrijdt `maxUnavailable`).

Waarom. De Deployment-controller past alleen de replica's van twee ReplicaSets aan, stap voor stap, binnen de twee grenzen en op basis van de toestand `Ready` van de Pods.

Nuance. De readiness probe geeft "klaar" zijn betekenis; zonder gaat de rollout sneller en kan hij de dienst onderbreken tijdens het opstarten van de JVM's.

Voorbeeld.

```
kubectl get events --sort-by=.lastTimestamp | grep ScalingReplicaSet | tail -6
```

Vraag 4 — Eén enkele Pod in `Error`

Antwoord. Het Deployment heeft **één** Pod van de nieuwe versie aangemaakt (`maxSurge` = 1) en wacht tot die klaar is voor het een oude weghaalt; omdat hij in een lus met een fout eindigt, **stopt de rollout daar**, en blijven de drie oude bedienen — dat is het vangnet. `kubectl rollout status` wacht en verloopt: `error: timed out waiting for the condition`. Oorzaak: `kubectl logs rs/api-6c87f96dd5 --tail=1` (of `kubectl logs api-6c87f96dd5-4v6s6 --previous`) → `demarrage impossible : configuration manquante`. Herstel: `kubectl rollout undo deployment/api`.

Waarom. `maxUnavailable: 25 %` van 3 = 0: geen enkele oude Pod mag weggehaald worden zolang er geen nieuwe klaar is.

Nuance. Zonder readiness probe had een container die **start** en enkele seconden later crasht, "klaar" kunnen zijn lang genoeg om een oude weg te halen; de sonde sluit dat venster.

Voorbeeld.

```
kubectl get deployment api      # READY 3/3   UP-TO-DATE 1   AVAILABLE 3
kubectl rollout undo deployment/api && kubectl rollout status deployment/api
```

Vraag 5 — Liveness op een afhankelijkheid

Antwoord. Wanneer de database uitvalt, wordt `/actuator/health` `DOWN`, faalt de liveness drie keer, en **doodt en herstart** de kubelet de API; de API herstart, vindt de database nog altijd niet, wordt opnieuw gedood: `RESTARTS` klimt, dan `CrashLoopBackOff`. Vijf minuten lang is de API, die zonder database verzoeken had kunnen bedienen (statische pagina's, cache, nette fouten), onbereikbaar, en bij de terugkeer van de database heeft ze meerdere minuten nodig om uit de backoff te komen. Er had een liveness op `/actuator/health/liveness` moeten staan (interne toestand van het proces, onafhankelijk van de afhankelijkheden) en, indien nodig, een **readiness** op de database: de Pod gaat uit het verkeer zonder gedood te worden.

Waarom. Herstarten herstelt alleen pannes **binnen** het proces; een afwezige afhankelijkheid herstel je niet met een herstart.

Nuance. Spring Boot scheidt de twee precies daarom: `LivenessState` hangt van geen enkele externe `HealthIndicator` af.

Voorbeeld.

```
kubectl describe pods -l app=api | grep -E 'Liveness probe failed|Killing' | tail -2
```

Vraag 6 — Gebroken readiness, gebroken liveness

Antwoord. Readiness op `8081`: één nieuwe Pod `0/1 Running` (nooit klaar), de drie oude `1/1`, `kubectl get deployment` toont `READY 3/3 UP-TO-DATE 1 AVAILABLE 3`; de rollout is geblokkeerd, de dienst **antwoordt** via de oude Pods. Liveness op `8081`: de nieuwe Pods worden `Ready` (de

readiness slaagt wel), de rollout **gaat door** en vervangt de drie oude, en dan doodt de liveness elke nieuwe Pod: `0/1 CrashLoopBackOff` × 3, dienst **dood**.

Waarom. De rollout wacht alleen op de readiness; de liveness werkt achteraf, op Pods die al als goed meegeteld zijn.

Nuance. Dat is het argument om een wijziging van een sonde in acceptatie te testen, en om de liveness een ruime `initialDelaySeconds` en `failureThreshold` te geven.

Voorbeeld.

```
kubectl get pods            # readiness: api-xxx 0/1 Running + 3 oude 1/1 ; liveness: 3 × CrashLoopBackOff
kubectl rollout undo deployment/api
```

Vraag 7 — `CrashLoopBackOff`

Antwoord. Een toestand van de **kubelet**: de container eindigt met een fout, de kubelet herstart hem, hij eindigt opnieuw, en de kubelet **verdubbelt de wachttijd** voor elke herstart (10 s, 20 s, 40 s... tot maximaal vijf minuten) om de node niet te verzadigen. De applicatie heeft maar één toestand: ze crasht. De oorzaak staat in `kubectl logs <pod> --previous`, de uitvoer van de vorige poging; `kubectl logs` zonder optie toont de **huidige** container, vaak leeg als hij net herstart is of nog niets geschreven heeft.

Waarom. De backoff is een bescherming van de node, geen diagnose: niets in de wachttijd zegt waarom het proces afsluit.

Nuance. De teller `RESTARTS` wordt op nul gezet wanneer de container tien minuten draait zonder af te sluiten: een oude herstart kan dus niet meer zichtbaar zijn.

Voorbeeld.

```
kubectl get pod api-xxx -o
jsonpath='{.status.containerStatuses[0].lastState.terminated.exitCode}'
kubectl logs api-xxx --previous | tail -3
```

Vraag 8 — `HEALTHCHECK` tegenover readiness

Antwoord. De `HEALTHCHECK` wordt **in** de container uitgevoerd door de engine (Podman via een timer, Docker via de daemon): de tool (`wget`, `curl`) moet in de image zitten; een mislukking verandert de toestand `(unhealthy)` en verder niets, behalve met Compose `depends_on`. De `readinessProbe httpGet` wordt door de **kubelet** uitgevoerd, vanaf de node, met een HTTP-verzoek naar het IP van de Pod: geen enkele tool in de image; een mislukking haalt de Pod uit het verkeer. Een distroless-image (zonder shell of `wget`) kan geen werkende `HEALTHCHECK` dragen; ze werkt perfect met een sonde `httpGet` of `tcpSocket`.

Waarom. Kubernetes heeft de controle van de container naar het platform verplaatst; daarom negeert het de `HEALTHCHECK` van images.

Nuance. De sonde `exec` neemt het Docker-model over (commando in de container) voor gevallen zonder HTTP-poort.

Voorbeeld.

```
kubectl explain pod.spec.containers.readinessProbe.httpGet | head -8
```

Vraag 9 — Revisies met gaten

Antwoord. Elke wijziging van het template maakt een revisie aan, en elke `rollout undo` ook: terugkeren naar revisie 1 **maakt** een revisie 3 aan die identiek is aan 1 (de ReplicaSet van 1 wordt hergebruikt en hernummerd). De ontbrekende nummers zijn die welke de terugkeren "opgebruikt" hebben: `1 → 3`, `2 → 4`, enzovoort. Geen probleem: de geschiedenis blijft coherent, en `rollout undo --to-revision=N` werkt met de getoonde nummers.

Waarom. Een revisie is een annotatie `deployment.kubernetes.io/revision` op een ReplicaSet; een opnieuw geactiveerde ReplicaSet krijgt het volgende nummer.

Nuance. De lege kolom `CHANGE-CAUSE` vul je met `kubectl annotate deployment api kubernetes.io/change-cause="overgang naar 1.1"`, waardevol bij een incidentreview.

Voorbeeld.

```
kubectl get rs -o custom-
columns=NAME:.metadata.name,REV:.metadata.annotations.deployment\.kubernetes\.io/revision,REPLI
CAS:.spec.replicas
```

Vraag 10 — `maxSurge` en `maxUnavailable`

Antwoord. `maxSurge: 0, maxUnavailable: 1`: het cluster haalt eerst een oude Pod weg en maakt dan een nieuwe aan; piek 3, dal **2** Pods — nooit meer dan drie, dus geschikt voor volle nodes, ten koste van minder capaciteit tijdens de update. `maxSurge: 1, maxUnavailable: 0`: het maakt een nieuwe Pod aan, wacht tot hij klaar is, en haalt dan een oude weg; piek **4**, dal 3 — nooit minder dan drie klaar, dus geschikt voor een dienst die geen capaciteit mag verliezen, ten koste van één Pod extra om te huisvesten.

Waarom. De twee grenzen lees je als een budget: hoeveel mag ik overschrijden, hoeveel mag ik missen.

Nuance. `maxSurge: 0, maxUnavailable: 0` wordt geweigerd: de rollout zou nooit kunnen beginnen.

Voorbeeld.

```
kubectl patch deployment api -p '{"spec":{"strategy":{"rollingUpdate":
{"maxSurge":1,"maxUnavailable":0}}}}'
```

Vraag 11 — Twee Pods in `Pending`

Antwoord. De node heeft geen **resources** meer voor hen (geheugen of CPU, als het template `requests` heeft, lab 16) of bereikt zijn limiet aan Pods; `kubectl describe pod` toont `FailedScheduling … 0/1 nodes are available: 1 Insufficient memory`. Een rollout in die toestand gaat **gedeeltelijk** door: de Pods die gepland kunnen worden, worden vervangen, de `Pending` blijven het (voor de nieuwe versie), en `rollout status` komt nooit tot een einde.

Waarom. De scheduler plaatst een Pod alleen als een node aan zijn vragen kan voldoen; `scale` schept geen capaciteit.

Nuance. Zonder `requests` aanvaardt een node Pods tot zijn aantalslimiet (standaard 110): de `Pending` komen dan van iets anders (volume, affiniteit, taint).

Voorbeeld.

```
kubectl describe node | grep -A6 'Allocated resources'
```

Vraag 12 — Een Pod met de hand wijzigen

Antwoord. Bij de volgende **rollout** wordt de Pod vervangen door een nieuwe Pod uit het template: de wijziging verdwijnt. Bij de volgende **herstart** van de container (liveness, crash) wordt het bestandssysteem van de writable layer opnieuw aangemaakt vanuit de image: ook verdwenen. De **twee andere replica's** hebben de wijziging nooit gezien: de dienst gedraagt zich anders naargelang de Pod die antwoordt. De configuratie hoorde in een **ConfigMap** of een **Secret** (lab 14) waarnaar het template verwijst, zodat de rollout de wijziging naar alle replica's verspreidt.

Waarom. Een Pod is wegwerpbaar (lab 06); alleen de spec is duurzaam.

Nuance. `kubectl exec` blijft legitiem om te **observeren** en voor een tijdelijke diagnose, nooit om te configureren.

Voorbeeld.

```
kubectl rollout restart deployment/api    # maakt alle Pods opnieuw aan vanuit het template: het geknutsel verdwijnt
```

Vraag 13 — Een kale Pod in productie

Antwoord. (1) **Geen zelfherstel**: verwijderd, of zijn node verloren, komt de Pod niet terug. (2) **Geen update zonder onderbreking**: de image veranderen vereist `delete` en dan `apply`. (3) **Geen terugkeer**: geen geschiedenis. (4) **Geen opschaling**: naar twee instanties gaan vraagt een tweede manifest. Het enige legitieme geval: een **eenmalige verwerking** of een test, wat een `Job` trouwens beter doet; voor een dienst, zelfs met één instantie, kost het Deployment maar vijf regels meer.

Waarom. De Pod is een uitvoeringseenheid, geen beheerobject; de controllers voegen het beheer toe.

Nuance. Een kale Pod wordt wél door de kubelet opnieuw aangemaakt als het een *static Pod* is (bestand op de node) — zo draaien de onderdelen van het control plane in sommige clusters, niet jouw applicaties.

Voorbeeld.

```
kubectl create deployment api --image=localhost/api:1.0 --replicas=1 --dry-run=client -o yaml | head -20
```